

Use of AI Tools

Human-Centric Artificial Intelligence, Projects 1–4

Kartik Anand — Matriculation 676049

Hamburg University of Technology SoSe 2026

September 5, 2026

Where an LLM was used in this project

Covers Projects 1 to 3. Updated as later projects are done.

The implementation was drafted with an LLM (Claude, via Claude Code) over a single working session, then read through and corrected. This file records which parts that applies to and where the line ran, since "AI-assisted" on its own says nothing useful.

By component

Part	How it was produced
Django wiring — <code>startapp</code> , <code>INSTALLED_APPS</code> , URL includes, forms, session handling	Drafted by the model. This is the same twelve steps every Django app needs and there is one correct way to do it.
Templates and CSS	Drafted by the model against a brief: keep the skeleton's blue-and-white identity, override the absolutely-positioned <code>.box</code> from the shared stylesheet because these pages are taller than the viewport.
Matplotlib figures	Drafted by the model. Choosing the four figures was a decision; getting the spines, palette and legend placement right afterwards was typing.
CSV loading, dropping bookkeeping columns	Rules decided first, then written up. See below.
Classification vs regression detection	Rule decided first, then written up. See below.
Fold construction, the sweep, the selection rule	Specified as an algorithm, then written up. See below.
Choice of models, hyperparameters and scores	Decided against the lecture material, not delegated.
Verification	Model-run: a script exercising every view and error path with Django's test client, plus a browser pass.

The parts the briefs reserve

Three components had more than one defensible answer, and those are the ones the brief is actually asking about.

To be accurate about how they were produced: the rules below were proposed by the model, under a standing instruction that ruled out the library defaults — **do not blindly use canned algorithms; for any algorithmic component where there are multiple reasonable choices, design and write the algorithm yourself based on the project requirements.** That instruction is the reason this project has a hand-written one-standard-error rule rather than a call to `GridSearchCV`, and it is the author's contribution to these three. The wording of each rule, and the argument for it, was then reviewed and kept.

Dropping an id column. Matching on the column name is the obvious approach and it is wrong — `patient_id` can be a real feature. The rule used here is structural: integer, unique across every row, and consecutive. A column that satisfies all three is numbering the rows.

Classification or regression. The common shortcut is a fixed cutoff, "ten or fewer distinct values means classes". That misreads a twelve-class problem and also misreads a small regression table. The cutoff here grows with the square root of the row count, floored at ten and capped at twenty, and the result is always shown on screen with an override available at upload.

Picking the hyperparameter. `GridSearchCV` is one line and would have done it. It also picks the argmax of five noisy numbers. What is implemented instead is a hand-rolled stratified fold split (deal each class's shuffled indices round-robin into k folds), an explicit sweep recording the mean, the standard error and the training-set score for every candidate, and a one-standard-error selection rule that prefers the plainer model among values statistically indistinguishable from the best. There is no scikit-learn equivalent for that last part. On the iris data every tree depth from 3 to 14 ties in cross-validation while the training score climbs to 1.0, which is the case the rule exists for.

Whether it was worth it

For the Django and presentation layers, clearly yes. That work is well-specified and has a known shape; time spent typing it out is time not spent on the part being graded.

For the three components above the answer is less comfortable. A model asked plainly to "pick a hyperparameter" reaches for `GridSearchCV`, which is exactly the answer the brief is trying to get past. What produced something better was the constraint, not the model: forbidding the canned answer forced a rule that had to be argued for. The lesson worth recording is that the quality of these came from the instruction given, and a project run without that constraint would have been visibly worse.

Project 2 specifically

The brief for Project 2 names parts that must be our own, and those are tracked formally rather than in prose: `home/own_work.py` holds each requirement with the brief's exact wording, the file satisfying it, and what was decided. Each such block in the code carries a marker comment, and `/home/own-work/` re-reads the files and reports whether the markers survive. That page fails loudly if a marked block is deleted or moved.

Requirement	What was decided, and by whom
Ω for logistic regression (Task 3)	Number of original features with a non-zero coefficient under L1. Decided first, then written up. The reasoning — that Ω must measure reader effort in both model classes, and that L2 would leave Ω constant — is the answer the brief is asking for, and it is not something to delegate.
Noising categorical data (§2)	Flip-with-probability, redrawn from the empirical distribution. Decided first.
Sparse counterfactual proposals	Mine, and not asked for. Perturbing all four measurements every draw made every answer useless; restricting each draw to a random subset turned "change all four things slightly" into "grow the bill by 4.5 mm".
The λ frontier	Mine, and not asked for. Recognising that $\text{acc} - \lambda\Omega$ is a line in λ , so only the upper concave hull is ever selectable, is a small piece of geometry that makes the slider explainable instead of mysterious. Model-drafted code, human-noticed idea.
PDP and ALE (Task 5)	Formulas taken from lecture 3 and the brief; the implementation written out. No library is called. The exact-vs-discretised distinction the brief asks about was reasoned through, not looked up.

The honest summary is the same as for Project 1: the model was fast at Django plumbing, matplotlib and templates, and would have reached for `sklearn.inspection.partial_dependence` for Task 5 — which is precisely the answer the brief exists to prevent.

Project 3 specifically

Two entries in the registry apply here: the simulated expert (Task 2) and the active learning strategy (Task 4), both marked in the code and both verified by the audit page.

Requirement	What was decided, and by whom
The expert's competence structure (Task 2)	Mine. The rule that competence must depend on the input and never on the true label is the decision the whole project rests on — an expert who is "reliable whenever the answer is Sports" cannot be discovered by querying, because the answer is exactly what is unknown at query time, so Task 4 would be vacuous.
The acquisition function (Task 4)	Mine. Targeting uncertainty in the *deferral decision* rather than in the classifier's prediction, on the grounds that the classifier is fixed and its competence is already known for free from the out-of-fold pass.
The feature representation	Settled by measurement, not preference — see below.
The evaluation	Mine. The four-case decomposition, and reporting deferral precision alongside accuracy, because accuracy alone rewards handing everything to a good expert.

I also ran a multi-agent design review at one point and it was a mistake worth recording. The agents had shell access and interpreted "design this" as "go and experiment", so they spent their budget computing lexical statistics on AG News and returned nothing usable; two of four died outright. The replacement — hand them the measurements and forbid running code — worked, but by then I had already settled the design from the measurements myself. The lesson is that an agent asked an open-ended design question will go looking for data, and if it has a shell it will find some.

Project 4 specifically

Tasks 1 and 2 are where this project's marks are, and both were produced under the same constraint as projects 1 to 3 rather than by asking for an answer.

Task 1's feature set is governed by the elicitation budget rather than by descriptive power: a genre earns a dimension only when $2p(1-p)$ clears one informative pair in ten, and director and cast are excluded as unidentifiable rather than weak. The threshold is computed from PAIR_INFORMATIVENESS in `project4/data.py` rather than hardcoded, so the prose and the code cannot drift apart.

Task 2 is the Plackett-Luce derivation and the argument for it. Worth being precise about what is easy here and what is not: an LLM will produce the Plackett-Luce formula on request, and reaching for it is no achievement. What carries the task is the observation that its pairwise marginals coincide with Bradley-Terry — the only reason scoring both study arms against a common held-out set is legitimate rather than a category error — and the specific rejection of the naive "split a ranking into all 45 pairs" alternative on the grounds that it double-counts dependent comparisons and does not sum to one over the $n!$ orderings. Both are verified numerically in `check_marginals` rather than asserted.

Drafting help was used for the plain-language explanation pages, the CSS and the prose of the report. Every quantitative claim in the report was produced by code in this repository and checked against it.

What Project 4's verification actually changed

The project began with the hypothesis that ranking ten films beats pairwise choice, which is what the per-task Fisher information says: one ranking carries about 60x one comparison. That hypothesis is wrong, and the simulation written to power the study is what showed it.

Two corrections stack. Fixing the participant's time rather than the task count means eight minutes buys 80 comparisons or 5 rankings, and the accumulated information traces then come out at 118 against 129 - a ratio of 1.09, not 60. And the trace turns out to be the wrong summary: over a full session the log-determinant is 17.6 nats lower for ranking and its smallest eigenvalue is 6.4 times smaller, because 80 comparisons touch 160 films while 5 rankings touch 50 and the 45 comparisons inside one ranking all lie in the span of that set's ten feature vectors. Ranking accumulates a lot of information about a few directions and leaves others to the prior.

The simulation confirms it at 120 simulated participants per condition: with ranking assumed flawless the two arms tie (paired $d = 0.00$), and with any position-dependent carelessness pairwise pulls clear ($d = -0.34$ and -0.51). H1 in the report was rewritten to the opposite direction, and the report says plainly that this is what pre-registration exists to make visible.

The honest reading is that the naive arithmetic was persuasive and wrong, and it would have gone into the report unchallenged if the power simulation had been skipped as a formality.

Why the framework layer was delegated

This is a deliberate allocation of effort, not an accident, and the reasoning is worth stating.

No brief asks for framework skill. The four briefs are about machine learning methodology and human-centric design. Django appears only as the delivery vehicle — project 1's own section on "useful tools" points at the demo app for file upload and at ChartJS as an alternative for plotting. It is scaffolding for the thing being assessed, not the thing being assessed.

The briefs name what they reserve, explicitly. Where own work is required they say so: `"the code for the computation of the PDP and ALE values should be written by you, i.e., do not use a library for them"`; `"choose a suitable complexity measure  $\Omega$ "`; `"design and implement at least one simulated expert"`; `"justify your choice"`. Nothing anywhere says anything comparable about URL routing or template inheritance. Those requirements are tracked in `home/own_work.py`, and the audit at `/home/own-work/` re-reads each named file and checks the imports, so the claim that no library did the reserved work is verified rather than asserted.

The delegated work has one correct shape. Registering an app, wiring `INSTALLED_APPS`, an URL include, a form, session state, saving a matplotlib figure to `MEDIA_ROOT` and serving it — every Django project does these the same way. There is no decision in them to get right or wrong, only typing.

The line is between plumbing and interface decisions, and only one side was delegated. Project 1's Task 4 asks `"what the user should be in control of, and what should be done automatically"`; project 1's Task 3 says `"the more you will do, the higher your grade"` about the interface. Those are graded, and they are design questions rather than framework questions: whether the app shows what it inferred, whether the user can override it, whether there is an automatic path to compare against. Those decisions are argued for in the implementation report. How a POST reaches a view is not.

One Django item is not delegable and was not treated as such. Project 1's Task 1

— `*"get familiar with this app logic? do this modification from the python, and not in the HTML!"*` — is a comprehension check. It is one line of code whose entire point is whether the view-context-template separation is understood, so it carries an own-work marker like the algorithmic requirements do.

The trade in one sentence. Hours spent typing framework code are hours not spent on the four things a marker can actually probe: why Ω counts non-zero coefficients rather than w ?, why a decision tree's ALE needs finite differences, why a simulated expert's competence must depend on x and not y , and why Plackett-Luce rather than a product over all 45 implied pairs.

What was checked by hand

- The stratified folds were checked to be disjoint, complete, and class-balanced.
- Project 2's frontier was checked against brute force: sweeping λ over 35,000

values and taking the arg max reproduces exactly the models the hull predicts.

- Project 2's two ALE routes were checked against each other; the closed-form

softmax derivative and the finite-difference estimate agree to $1.5e-4$.

- The PDP was checked against an independent hand computation.
- Project 3's classifier scored 3,403 errors in-sample against 11,350

out-of-fold. Every downstream target uses the out-of-fold pass; the in-sample figure would have understated classifier failure by 3.3x and taught the deferral model that the classifier almost never needs help.

- Project 3's feature representation was chosen by measurement: the classifier's

own top-two margin predicts its correctness at AUC 0.848, beating all 50,000 TF-IDF features (0.801). The compact SVD-100 + confidence representation scores 0.855 in 103 dimensions, which is what makes a few hundred expert queries enough to fit anything at all.

- Project 3's headline result was checked for the null case and one was found and

kept: the second expert shows no separation between any query strategy. That narrows the claim from "choosing questions beats random" to "choosing questions helps when the expert's competence is visible in the available features".

- Four bugs surfaced during verification of Project 1 and were fixed: a misleading error

message on very small files, a crash when the default k grid exceeded the rows available in a fold, a hardcoded claim in the AutoML warning text that happened to be true for iris and false in general, and a 500 when a text target was forced to be a regression problem.

The last of those is the one worth remembering. The override had been tested, but only on a numeric target, where it happens to work. A test that exercises a feature on the one input it

was written against is not a test. The override is now checked across every combination of the three problem-type settings and three shapes of target column.

Two more were found in Project 2. The `saga` solver is stochastic, and without a fixed `random_state` the fitted coefficients — and therefore Ω , and therefore the whole frontier — changed between runs; caught by running the same computation in three separate processes and comparing. And `text-transform: uppercase` turns a Greek lambda into a capital lambda, which reads as an "A", so the slider label said "REGULARISATION A" until it was noticed on screen.

Project 4 added three more. `enumitem` is absent from this TeX Live basic install and cannot be added without admin rights, so the two list styles it provided were rebuilt from the base classes - and the rebuild was briefly recursive, because a blanket replacement of `\end{enumerate}` also hit the one inside the new environment's own definition. An em dash written as `\{\}\u2014` in a patch script landed in `explain.py` as the literal characters, caught by grepping the loaded topics for a stray backslash-u rather than by reading the page. And the first version of `score_holdout` counted an exactly-even prediction as a miss, which scored an untrained model at 0% instead of 50% and would have flattered every fit it was compared against.